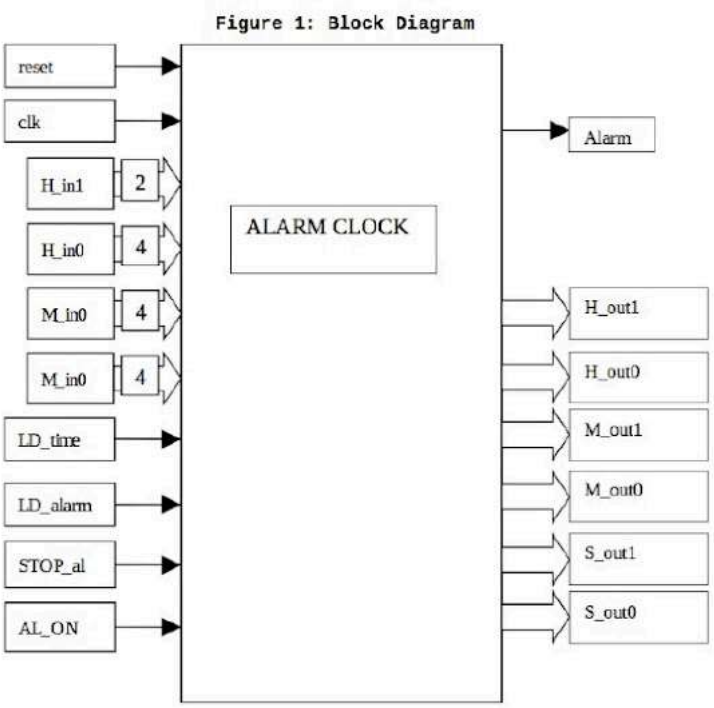
Verilog code for Alarm clock on FPGA

Verilog code for an alarm clock on FPGA is presented in this project. The Verilog code is fully synthesizable for FPGA implementation.
The simple alarm clock is shown in the following figure. The alarm clock outputs a real-time clock with a 24-hour format and also provides an alarm feature. Users also can set the clock time through switches.



Full Verilog code for the alarm clock:

```
// fpga4student.com FPGA projects, VHDL projects, Verilog project
module aclock (
 input reset,  /* Active high reset pulse, to set the time to the input hour and m
 input clk,  /* A 10Hz input clock. This should be used to generate each real-time
 input [1:0] H_in1, /*A 2-bit input used to set the most significant hour digit of
 input [3:0] H_in0, /* A 4-bit input used to set the least significant hour digit
 input [3:0] M_in1, /*A 4-bit input used to set the most significant minute digit
 input [3:0] M_in0, /*A 4-bit input used to set the least significant minute digit
 input LD_time,  /* If LD_time=1, the time should be set to the values on the inpu
 input   LD_alarm,  /* If LD_alarm=1, the alarm time should be set to the values o
 input   STOP_al,  /* If the Alarm (output) is high, then STOP_al=1 will bring the
 input   AL_ON,  /* If high, the alarm is ON (and Alarm will go high if the alarm
 output reg Alarm,  /* This will go high if the alarm time equals the current time
 output [1:0]  H_out1,
/* The most significant digit of the hour. Valid values are 0 to 2. */
 output [3:0]  H_out0,
/* The least significant digit of the hour. Valid values are 0 to 9. */
 output [3:0]  M_out1,
/* The most significant digit of the minute. Valid values are 0 to 5.*/
 output [3:0]  M_out0, /* The least significant digit of the minute. Valid values
 output [3:0]  S_out1, /* The most significant digit of the minute. Valid values a
 output [3:0]  S_out0  /* The least significant digit of the minute. Valid values
 );
// fpga4student.com FPGA projects, VHDL projects, Verilog project
 // internal signal
 reg clk_1s; // 1-s clock
```

```
 reg [3:0] tmp_1s; // count for creating 1-s clock
 reg [5:0] tmp_hour, tmp_minute, tmp_second;
// counter for clock hour, minute and second
 reg [1:0] c_hour1,a_hour1;
/* The most significant hour digit of the temp clock and alarm. */
 reg [3:0] c_hour0,a_hour0;
/* The least significant hour digit of the temp clock and alarm. */
 reg [3:0] c_min1,a_min1;
/* The most significant minute digit of the temp clock and alarm.*/
 reg [3:0] c_min0,a_min0;
/* The least significant minute digit of the temp clock and alarm.*/
 reg [3:0] c_sec1,a_sec1;
/* The most significant second digit of the temp clock and alarm.*/
 reg [3:0] c_sec0,a_sec0;
/* The least significant minute digit of the temp clock and alarm.*/

 /************************************************/
 /*****************function mod10******************/
 /************************************************/
 function [3:0] mod_10;
 input [5:0] number;
 begin
 mod_10 = (number >=50) ? 5 : ((number >= 40)? 4 :((number >= 30)? 3 :((number >=
 end
 endfunction

 /************************************************/
 /************* Clock operation**************/
 /************************************************/
// fpga4student.com FPGA projects, VHDL projects, Verilog project
 always @(posedge clk_1s or posedge reset )
 begin
 if(reset) begin // reset high => alarm time to 00.00.00, alarm to low, clock to H
 a_hour1 <= 2'b00;
 a_hour0 <= 4'b0000;
 a_min1 <= 4'b0000;
 a_min0 <= 4'b0000;
 a_sec1 <= 4'b0000;
 a_sec0 <= 4'b0000;
 tmp_hour <= H_in1*10 + H_in0;
 tmp_minute <= M_in1*10 + M_in0;
 tmp_second <= 0;
 end
 else begin
 if(LD_alarm) begin // LD_alarm =1 => set alarm clock to H_in, M_in
 a_hour1 <= H_in1;
 a_hour0 <= H_in0;
 a_min1 <= M_in1;
 a_min0 <= M_in0;
 a_sec1 <= 4'b0000;
 a_sec0 <= 4'b0000;
 end
 if(LD_time) begin // LD_time =1 => set time to H_in, M_in
 tmp_hour <= H_in1*10 + H_in0;
 tmp_minute <= M_in1*10 + M_in0;
 tmp_second <= 0;
 end
 else begin  // LD_time =0 , clock operates normally
 tmp_second <= tmp_second + 1;
 if(tmp_second >=59) begin // second > 59 then minute increases
 tmp_minute <= tmp_minute + 1;
 tmp_second <= 0;
 if(tmp_minute >=59) begin // minute > 59 then hour increases
```

```
 tmp_minute <= 0;
 tmp_hour <= tmp_hour + 1;
 if(tmp_hour >= 24) begin // hour > 24 then set hour to 0
 tmp_hour <= 0;
 end
 end
 end

 end
 end
 end

 /***************************************************/
 /******** Create 1-second clock****************/
 /***************************************************/
// fpga4student.com FPGA projects, VHDL projects, Verilog project
 always @(posedge clk or posedge reset)
 begin
 if(reset)
 begin
 tmp_1s <= 0;
 clk_1s <= 0;
 end
 else begin
 tmp_1s <= tmp_1s + 1;
 if(tmp_1s <= 5)
 clk_1s <= 0;
 else if (tmp_1s >= 10) begin
 clk_1s <= 1;
 tmp_1s <= 1;
 end
 else
 clk_1s <= 1;
 end
 end

 /***************************************************/
 /***OUTPUT OF THE CLOCK*********************/
 /***************************************************/
 always @(*) begin

 if(tmp_hour>=20) begin
 c_hour1 = 2;
 end
 else begin
 if(tmp_hour >=10)
 c_hour1  = 1;
 else
 c_hour1 = 0;
 end
 c_hour0 = tmp_hour - c_hour1*10;
 c_min1 = mod_10(tmp_minute);
 c_min0 = tmp_minute - c_min1*10;
 c_sec1 = mod_10(tmp_second);
 c_sec0 = tmp_second - c_sec1*10;
 end
 // fpga4student.com FPGA projects, VHDL projects, Verilog project
 assign H_out1 = c_hour1; // the most significant hour digit of the clock
 assign H_out0 = c_hour0; // the least significant hour digit of the clock
 assign M_out1 = c_min1; // the most significant minute digit of the clock
 assign M_out0 = c_min0; // the least significant minute digit of the clock
 assign S_out1 = c_sec1; // the most significant second digit of the clock
 assign S_out0 = c_sec0; // the least significant second digit of the clock
```

```
 /***************************************************/
 /******** Alarm function*****************/
 /***************************************************/
// fpga4student.com FPGA projects, VHDL projects, Verilog project
 always @(posedge clk_1s or posedge reset) begin
 if(reset)
 Alarm <=0;
 else begin
 if({a_hour1,a_hour0,a_min1,a_min0,a_sec1,a_sec0}={c_hour1,c_hour0,c_min1,c_min0,
 begin // if alarm time equals clock time, it will pulse high the Alarm signal wit
 if(AL_ON) Alarm <= 1;
 end
 if(STOP_al) Alarm <=0; // when STOP_al = 1, push low the Alarm signal
 end
 end

endmodule
```

Testbench Verilog code for alarm clock:

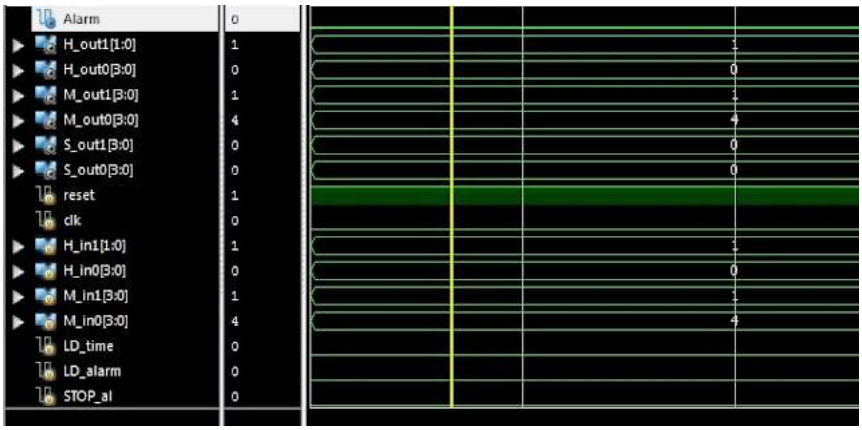
```
module test;
// fpga4student.com FPGA projects, VHDL projects, Verilog project
 // Inputs
 reg reset;
 reg clk;
 reg [1:0] H_in1;
 reg [3:0] H_in0;
 reg [3:0] M_in1;
 reg [3:0] M_in0;
 reg LD_time;
 reg LD_alarm;
 reg STOP_al;
 reg AL_ON;

 // Outputs
 wire Alarm;
 wire [1:0] H_out1;
 wire [3:0] H_out0;
 wire [3:0] M_out1;
 wire [3:0] M_out0;
 wire [3:0] S_out1;
 wire [3:0] S_out0;
// fpga4student.com FPGA projects, VHDL projects, Verilog project
 // Instantiate the Unit Under Test (UUT)
 aclock uut (
 .reset(reset),
 .clk(clk),
 .H_in1(H_in1),
 .H_in0(H_in0),
 .M_in1(M_in1),
 .M_in0(M_in0),
 .LD_time(LD_time),
 .LD_alarm(LD_alarm),
 .STOP_al(STOP_al),
 .AL_ON(AL_ON),
 .Alarm(Alarm),
 .H_out1(H_out1),
 .H_out0(H_out0),
 .M_out1(M_out1),
 .M_out0(M_out0),
 .S_out1(S_out1),
 .S_out0(S_out0)
 );
```

```
// clock 10Hz
initial begin
 clk = 0;
 forever #50 clk = ~clk;
end
initial begin
// Initialize Inputs
reset = 1;
H_in1 = 1;
H_in0 = 0;
M_in1 = 1;
M_in0 = 4;
LD_time = 0;
LD_alarm = 0;
STOP_al = 0;
AL_ON = 0; // set clock time to 11h26, alarm time to 00h00 when reset
// Wait 100 ns for global reset to finish
#1000;
     reset = 0;
H_in1 = 1;
H_in0 = 0;
M_in1 = 2;
M_in0 = 0;
LD_time = 0;
LD_alarm = 1;
STOP_al = 0;
AL_ON = 1; // turn on Alarm and set the alarm time to 11h30
#1000;
reset = 0;
H_in1 = 1;
H_in0 = 0;
M_in1 = 2;
M_in0 = 0;
LD_time = 0;
LD_alarm = 0;
STOP_al = 0;
AL_ON = 1;
wait(Alarm); // wait until Alarm signal is high when the alarm time equals clock
#1000
STOP_al = 1; // pulse high the STOP_al to push low the Alarm signal
#1000
STOP_al = 0;
H_in1 = 0;
H_in0 = 4;
M_in1 = 4;
M_in0 = 5;
LD_time = 1; // set clock time to 11h25
LD_alarm = 0;
#1000
STOP_al = 0;
H_in1 = 0;
H_in0 = 4;
M_in1 = 5;
M_in0 = 5;
LD_alarm = 1; // set alarm time to 11h35
LD_time = 0;
wait(Alarm); // wait until Alarm signal is high when the alarm time equals clock
#1000
STOP_al = 1;// pulse high the STOP_al to push low the Alarm signal
// fpga4student.com FPGA projects, VHDL projects, Verilog project
end

endmodule
```

This Verilog code for the alarm clock is simulated using Xilinx ISIM. Simulation waveform for the alarm clock is as follows:



VHDL code for a digital clock
What is FPGA Programming? FPGA vs Software programming
Recommended and affordable Xilinx FPGA boards for students
Recommended and affordable Altera FPGA boards for students
Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**
36. **Verilog code for Moore FSM Sequence Detector**
37. **Verilog code for 7-segment display controller on Basys 3 FPGA**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.